Important Conditional Branch Instructions

Instruction	Meaning
BEQ	Branch if Equal
BNE	Branch if Not Equal
BC	Branch if Carry
BNC	Branch if No Carry
BZ	Branch if Zero
BNZ	Branch if Not Zero
BM	Branch if Minus
BP	Branch if Plus
BGT	Branch if Greater then
BLT	Branch if Less then
BGE	Branch if Greater then or equal
BLE	Branch if Less then or equal

1. BEQ (BRANCH IF EQUAL)

```
1    MOV R1, 10
2    MOV R2, 10
3    CMP R1, R2
4    BEQ EQUAL
5    MOV R3, 0

 EQUAL:
1   ADD R1,R2
2    HALT
```

2. Branch if Not Equal – BNE

```
1    MOV R1, 10
2    MOV R2, 20
3    CMP R1, R2
4    BNE NOTEQUAL
5    MOV R3, 1

NOTEQUAL:
1    SUB R2, R1
2    STORE C , R2
3    HALT
```

3. Branch if Zero – BZ

Question: Check whether a register contains zero.

```
1    MOV R1, 0
2    CMP R1, 0
3    BZ ZERO
4    MOV R2,
ZERO:
1    MOV R2, 0
2    HALT
```

4. Branch if Not Zero – BNZ

```
1    MOV R1, 5
2    CMP R1, 0
3    BZ ZERO
4    MOV R2,
ZERO:
1    ADD R1, 20
2    HALT
```

5. Branch if Greater Than – BGT

Question: Find the greater of two numbers.

```
1    MOV R1, 20
2    MOV R2, 10
3    CMP R1, R2
4    BGT R1GREATER
5    MOV R3, R2
R1GREATER:
1    MOV R3, R1
2    HALT
```

6. Branch if Less Than – BLT

```
1    MOV R1, 5
2    MOV R2, 10
3    CMP R1, R2
4    BLT R1LESS
5    MOV R3, R2
R1LESS:
1    MOV R3, R1
2    HALT
```

7. Branch if Carry – BC

```
1    MOV R1, 255
2    MOV R2, 1
3    ADD R1, R2
4    BC CARRY
5    MOV R3, 0
CARRY:
1    MOV R3, 1
2    HALT
```

8. Branch if No Carry – BNC

```
1    MOV R1, 10
2    MOV R2, 20
3    ADD R1, R2
4    BNC NOCARRY
5    MOV R3, 0
NOCARRY:
1    MOV R3, 1
2    HALT
```

BM

```
MOV R1, A
CMP R1, 0
BM NEGATIVE

Number is not negative
MOV R2, 0
JMP END

NEGATIVE:
MOV R2, 1

END:
MOV RESULT, R2
```

Find Whether the Difference is Negative

```
MOV R1, A
MOV R2, B
SUB R1, R2
BM NEGATIVE

MOV R2, 0
JMP END

NEGATIVE:
   MOV R2, 1

END:
   MOV RESULT, R2
```

Addition of Two Numbers Without Considering Carry

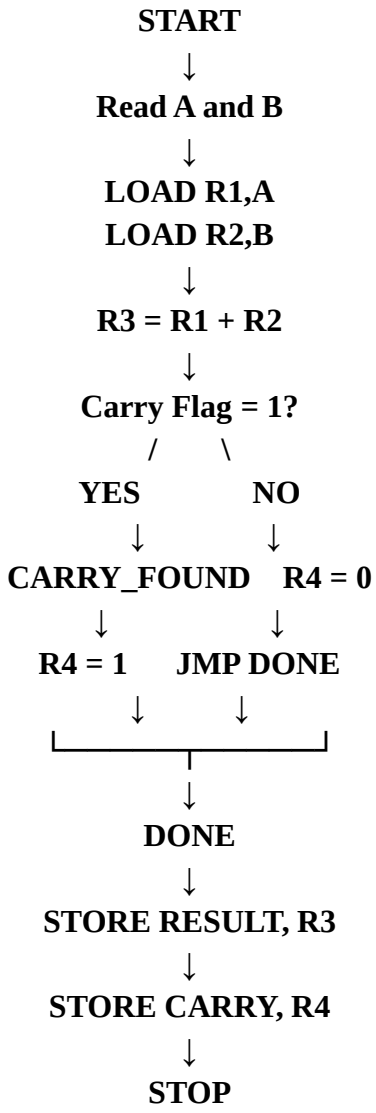
Two numbers: A = 10, B = 20

Result = 10 + 20 = 30 ----------> there is no carry

```
LOAD R1, A                    ; Load A into R1
LOAD R2, B                    ; Load B into R2
ADD  R3, R1, R2               ; R3 = R1 + R2
STORE C, R3                   ; Store result in C
```

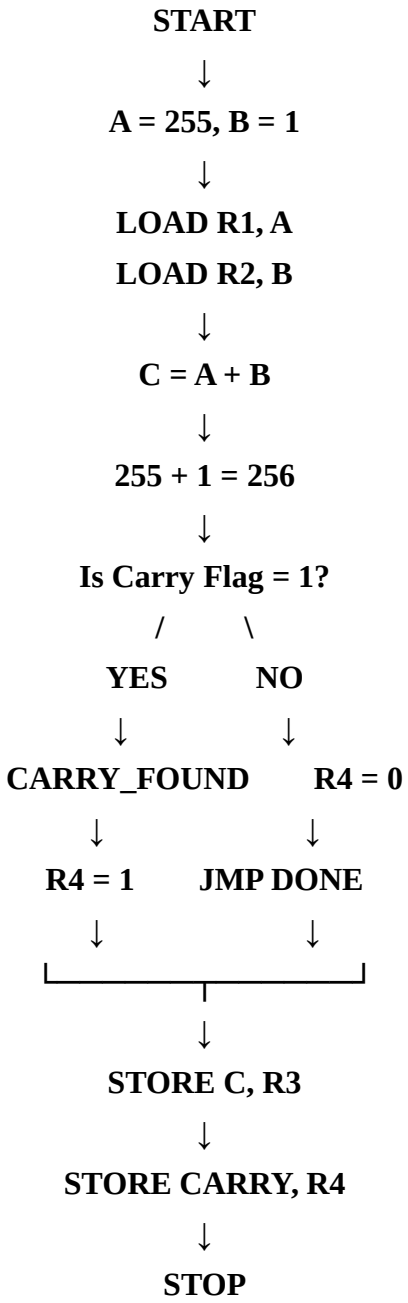
Addition of Two Numbers With Considering Carry

```
LOAD R1, A                ; R1 = A
LOAD R2, B                ; R2 = B
ADD  R3, R1, R2           ; R3 = R1 + R2
                          ; Carry Flag may become 1
BC   CARRY_FOUND          ; Branch if Carry = 1
MOV  R4, #0               ; No carry
JMP  DONE
CARRY_FOUND:
MOV  R4, #1               ; Carry occurred
DONE:
STORE RESULT, R3
STORE CARRY, R4
```



A = 255, B = 1, C = A + B

```
LOAD R1, A              ; R1 = 255
LOAD R2, B              ; R2 = 1
ADD  R3, R1, R2         ; R3 = R1 + R2
                        ; 255 + 1 = 256
                        ; 8-bit result = 0
                        ; Carry = 1
BC   CARRY_FOUND        ; If Carry = 1, branch
MOV  R4, #0             ; No carry
JMP  DONE               ; Skip carry section
CARRY_FOUND:
MOV  R4, #1             ; Carry occurred
DONE:
STORE C, R3             ; C = 0
STORE CARRY, R4         ; CARRY = 1
```



Addition of N Numbers Using an Array

```
int A[5] = {10, 20, 30, 40, 50};
int n = 5;
int sum = 0;
int i;
for(i = 0; i < n; i++) {
sum = sum + A[i];}
printf("Sum = %d", sum);
return 0;
```

Initially sum = 0

i	A[i]	sum = sum + A[i]
0	10	0 + 10 = 10
1	20	10 + 20 = 30
2	30	30 + 30 = 60
3	40	60 + 40 = 100
4	50	100 + 50 = 150

sum 150

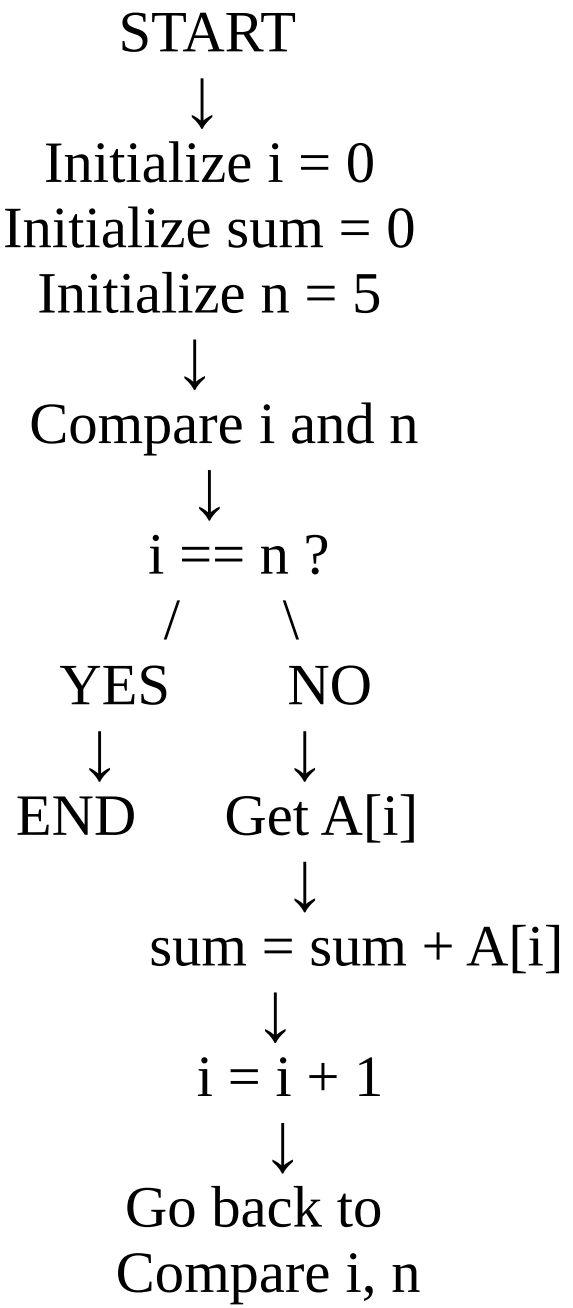
```
        MOV R1, 0          ; i = 0
        MOV R2, 5          ; n = 5
        MOV R3, 0          ; sum = 0

LOOP:   CMP R1, R2         ; compare i and n
        BEQ DONE           ; if i == n, exit loop

        LOAD R5, A[R1]     ; R5 = A[i]
        ADD R3, R3, R5     ; sum = sum + A[i]

        ADD R1, R1, 1      ; i = i + 1
        JMP LOOP           ; repeat

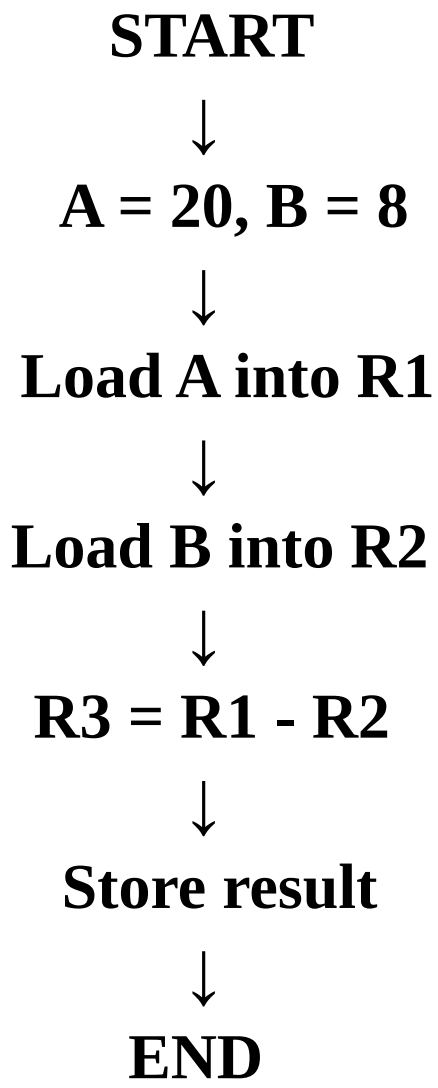
DONE:   MOV R6, R3         ; R6 = final sum
```



Subtraction of two numbers

Let A = 20, B = 8, C = A - B

```
MOV R1, 20        ; R1 = 20
MOV R2, 8         ; R2 = 8
SUB R3, R1, R2    ; R3 = R1 - R2
MOV R4, R3        ; R4 = result
```

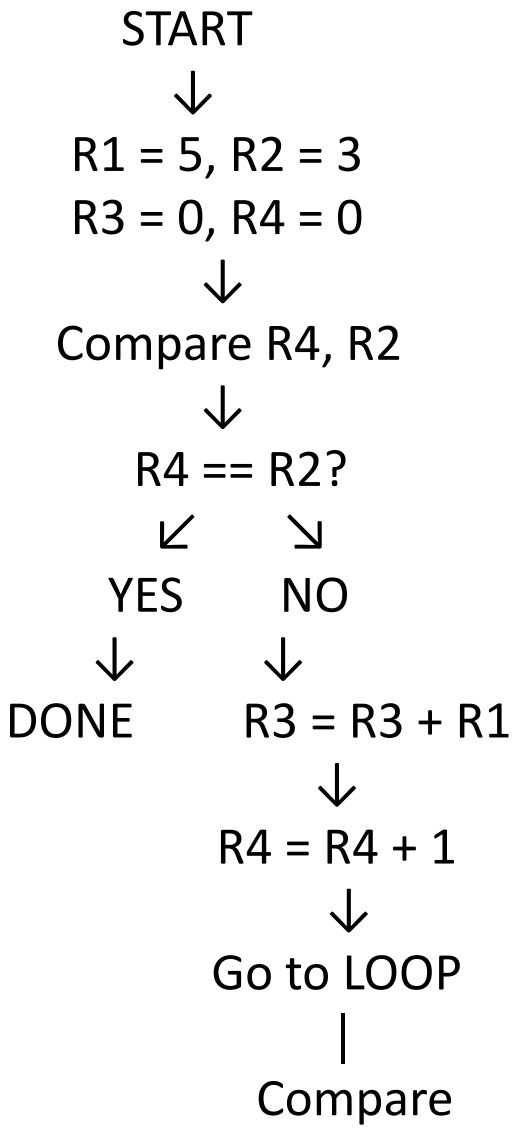


Multiplication with repeated addition

Let a = 5, b = 3

```
        MOV R1, 5              ; R1 = 5
        MOV R2, 3              ; R2 = 3
        MOV R3, 0              ; result = 0
        MOV R4, 0              ; counter = 0
LOOP:   CMP R4, R2             ; compare counter with 3
        BEQ DONE               ; if counter == 3, stop
        ADD R3, R3, R1         ; result = result + 5
        ADD R4, R4, 1          ; counter = counter + 1
        JMP LOOP               ; repeat
DONE:   MOV R5, R3             ; R5 = final result
```

Iteration	R3 (Result)	R4 Counter
Initial	**0**	**0**
1st	**0 + 5 = 5**	**1**
2nd	**5 + 5 = 10**	**2**
3rd	**10 + 5 = 15**	**3**

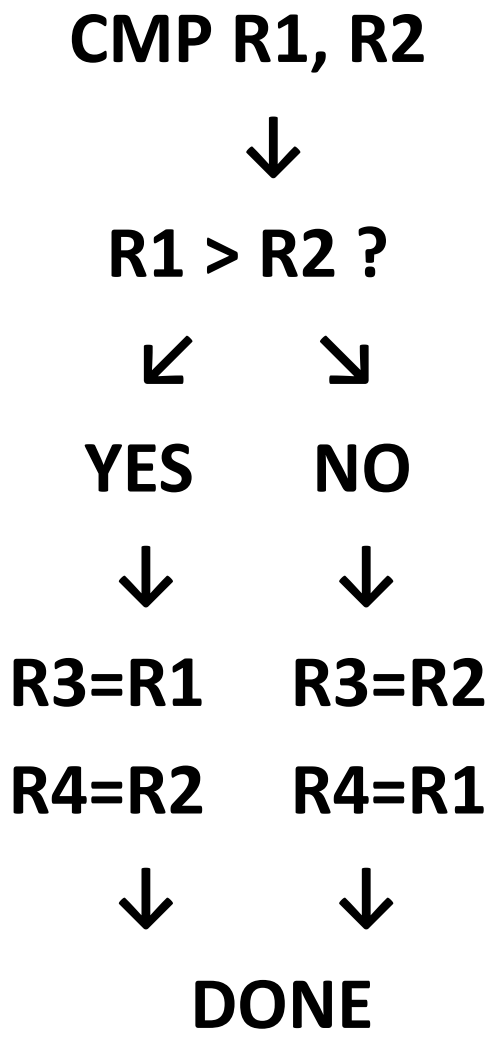


Find the greatest number and smallest number between two numbers

Let A = 20, B = 8

```
    MOV R1, 20              ; R1 = 20
    MOV R2, 8               ; R2 = 8
    CMP R1, R2              ; Compare R1 and R2
    BGT FIRST_GREATER       ; If R1 > R2, go to FIRST_GREATER
    MOV R3, R2              ; R2 is greatest
    MOV R4, R1              ; R1 is smallest
    JMP DONE                ; Go to DONE
FIRST_GREATER:
    MOV R3, R1              ; R1 is greatest
    MOV R4, R2              ; R2 is smallest

DONE:
```



Find the largest no from a given array

Array = [10, 25, 15, 40, 30]

R3 = 10

Let, assume first element is largest

compare one by one:

10 vs 25 → 25 is larger → R3 = 25

25 vs 15 → 25 is larger → R3 = 25

25 vs 40 → 40 is larger → R3 = 40

40 vs 30 → 40 is larger → R3 = 40

R5 = 40

```
        MOV R1, 0               ; R1 = index i
        MOV R2, 5               ; R2 = number of elements
        LOAD R3, A[0]           ; R3 = largest = first element
                                ; A[0] = 10, R3 = 10
LOOP:   ADD R1, R1, 1           ; i = i + 1
        CMP R1, R2              ; compare i with n
        BEQ DONE                ; if i == n, stop

        LOAD R4, A[R1]          ; R4 = A[i]
        CMP R4, R3          ; compare current element with largest
        BLE LOOP                ; if R4 <= R3, go to LOOP

        MOV R3, R4              ; R3 = new largest
        JMP LOOP                ; repeat

DONE:
        MOV R5, R3              ; R5 = largest number
```

